

CONVERSATIONAL SHL ASSESSMENT RECOMMENDER

Approach

I built a FastAPI service with GET /health and POST /chat. The chat endpoint accepts a list of user/assistant messages and returns a JSON response containing a reply, zero or more SHL recommendations, and an end_of_conversation flag. The service supports clarifying questions for vague requests, off-topic refusal, relevant recommendations, simple comparison requests, and result refinement through the conversation history.

Retrieval and Grounding

The catalog is loaded from data/catalog.json, which was normalized from the official SHL product catalog JSON linked in the assignment PDF. Each assessment record includes name, URL, remote-testing support, adaptive/IRT support, test type metadata, duration, job levels, languages, and description where available. A lightweight retrieval layer scores catalog items against role and skill terms such as Java, Python, SQL, React, numerical reasoning, verbal reasoning, personality, motivation, and mechanical comprehension. Retrieved catalog rows are passed to the LLM as evidence. A hallucination guard validates every returned URL against the catalog and falls back to catalog names when possible.

Prompt Design

The system prompt instructs the model to behave as an SHL Assessment Recommender, ask exactly one clarifying question for vague requests, refuse off-topic questions, recommend only catalog assessments, compare using only catalog metadata, and return valid JSON only. The response schema is enforced by FastAPI/Pydantic and a post-processing validation layer.

LLM

The implementation uses the Google GenAI SDK. The model is configurable with GEMINI_MODEL and defaults to gemini-2.0-flash. The API key is read from GEMINI_API_KEY in a root-level .env file. If Gemini is unavailable or quota-limited, the service falls back to deterministic catalog retrieval so the API remains usable.

Evaluation

I added evaluation/eval_cases.json and evaluation/evaluate.py. The evaluator sends representative chat requests through the API and measures retrieval quality, recommendation relevance, groundedness, and behavior accuracy. Metrics include recall against expected assessments, precision of returned recommendations, groundedness by checking all URLs against the catalog, and task behavior checks for clarify/refuse/recommend/compare flows.

Results

Current evaluation over six cases using the official 377-item catalog: mean recall 1.000, mean precision 0.592, groundedness 1.000, and behavior accuracy 1.000. Precision is lower than the small prototype catalog because broad role requests intentionally return several related SHL options, up to the assignment limit of 10 recommendations. Unit tests also cover health, vague queries, Java recommendations, turn-cap behavior, and off-topic refusal.

What Did Not Work

The original virtual environment used Python 3.14, which failed to build the pinned pydantic-core dependency. I created a Python 3.12 environment. The original deployment config also referenced requirements.txt while the repo only had requirement.txt, so I added requirements.txt. A hardcoded older Gemini model failed, so the model is now configurable and defaults to gemini-2.0-flash. The root URL originally returned 404, so it now redirects to /docs.

Page 2